

Reprography Automation System

Project Overview

Reprography Automation System is a full-stack web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that completely digitises and automates a college campus reprography centre. It replaces manual, paper-based processes with a structured, intelligent, and fully trackable digital workflow covering every service offered at a physical print shop - printing, photocopying, scanning, binding, document conversion, image and PDF processing - under one platform.

Users submit service requests online, upload their documents, make payments, and track order status in real time from any device - without visiting the counter until their output is ready for pickup. Counter staff at the physical shop process orders from a dedicated operational dashboard. The admin manages the entire system - pricing, users, analytics, inventory, and finances - from a separate management interface. Real-time communication between all three roles is handled through Socket.io WebSockets.

Sl. No.	Feature	Category
1	Authentication (JWT + Bcrypt)	Core
2	Three-Tier Role System with Composite Priority Queue	Core
3	Document Upload with AI Analysis	Core + AI
4	Document Processing Services + Per-Order Print Instructions	Core
5	Document Tools – Free Utility Suite	Utility

6	Multi-Document Orders	Enhancement
7	Smart Cost Estimation with AI Optimisation Suggestion	Core + AI
8	Payment System (Razorpay, Cash, Retry, Coupons, Receipts)	Core
9	Order Editing and Cancellation	Core
10	Order Tracking with Real-Time Status Timeline	Core
11	Partial Order Handling	Enhancement
12	Slot Booking and Preferred Pickup Time	Enhancement
13	Counter Staff Dashboard	Operations
14	Admin Dashboard	Management
15	Admin Activity Logs (Append-Only Audit Trail)	Management
16	Inventory Tracker with Demand Forecasting	Operations
17	Reports and Analytics (9 Report Types)	Management
18	Notifications with Persistent History	Core

19	Order History and Receipt Generation	Core
20	Complaint and Messaging System (Real-Time Chat)	Support
21	AI Chatbot Assistant (Gemini API + Live DB Context)	AI
22	Search and Filtering – All System Tables	Core
23	User Feedback and Rating System	Enhancement
24	Backup and Recovery Strategy	Infrastructure
25	Print Instructions (Per-Page / Free Text)	Core
26	Operating Hours and Shop Open/Closed Status	Core
27	Reorder – Repeat a Previous Order	Core
28	Document Preview Before Payment	Core
29	Pickup Verification via OTP	Core + Security
30	Maximum Order Size Limits	Core
31	User Profile Page with Session Management	Core

32	Email Verification at Registration	Core
33	Pay at Counter – Unpaid, Uncollected and Disputed Handling	Core
34	Staff Operational Reporting System	Operations
35	Security – 6 Categories, 30+ Independent Controls	Core
36	Decentralised Smart Kiosk Mode (Guest Session)	Innovation
37	PWA Architecture – Installable App, Offline Fallback, Service Worker Push Notifications	Mobile / Infrastructure
38	Friend Chat System – Friends, Direct Chat, Group Chat, Media, Voice, Read Receipts	Communication
39	Split Requests and Live Split Cards – Group-Order Settlement Through Wallet Transactions	Group Payments / Wallet

Stack Layer	Technology
Frontend	React.js, Tailwind CSS, Socket.io Client, Chart.js
Backend	Node.js, Express.js, Socket.io Server

Database	MongoDB with Mongoose ODM
Stack Layer	Technology

File Storage	Cloudinary - cloud-based with signed URL access control
Payments	Razorpay - UPI, Card, Net Banking, Wallet
Authentication	JWT (JSON Web Tokens) + Bcrypt (salt factor 12)
AI and Document Processing	Gemini API, sharp, pdf-lib, pdfjs-dist
Real-Time	Socket.io WebSockets
Security	Helmet, CORS, Rate Limiting, HMAC-SHA256, MIME validation

Three-Tier Role Architecture

RAS is built on a three-tier role model. Each role has a clearly defined scope of access enforced independently at both the frontend routing level and the backend API middleware level. Bypassing the frontend does not grant backend access.

Role	Who They Are	Primary Responsibility
User	Students and Teaching Faculty	Place orders, pay, track status, use document tools, raise complaints
Counter Staff	Physical print shop workers at the	reprography counter Process queue orders, update status, collect cash, manage inventory, respond to complaints
Admin	Reprography centre manager	Full system oversight - analytics, pricing, user accounts, finances, reports, refunds, audit logs

Counter Staff accounts are created exclusively by the Admin - self-registration is not permitted for this role. Admin accounts are system-seeded and cannot be self-registered by any user. This is a deliberate security decision: an attacker who registers cannot grant themselves elevated privileges.

Capability	User	Counter Staff	Admin
Place orders and make payments	Yes	No	No
Track own orders and view history	Yes	No	No
Use document tools (free utilities)	Yes	No	No
Raise and chat on complaints	Yes	No	No
View live queue	No	Yes	Yes

Capability	User	Counter Staff	Admin
Process orders and update status	No	Yes	Yes
Collect cash payments at counter	No	Yes	Yes
Respond to complaints	No	Yes	Yes
Update inventory stock levels	No	Yes	Yes
View analytics and reports	No	No	Yes
Manage service pricing	No	No	Yes
Manage user accounts	No	No	Yes
View all payment transactions	No	No	Yes
Approve and trigger refunds	No	No	Yes
Create Counter Staff accounts	No	No	Yes
Override queue positions	No	No	Yes
View admin activity logs	No	No	Yes

Feature Descriptions

1. Authentication

What it actually does: When someone opens RAS for the first time they see a login page. They click Register and fill in their name, email, college ID, department, and role - Student or Teacher/Faculty. Counter Staff accounts cannot be self-registered; they are created by the Admin only. Passwords are hashed using Bcrypt with a salt factor of 12 before storage, making reverse-engineering computationally infeasible even if the database is breached.

After login the server generates a JWT token containing the user's ID and role. This token is stored on the client and sent with every API request. Every protected route validates the token through middleware - an expired, missing, or tampered token returns a 401 response and the user is redirected to the login page. Role verification runs as a completely independent second layer - a valid student token hitting an admin-only endpoint returns a 403 response regardless of the token's validity.

A forgot password flow sends a time-limited reset link to the user's email. Clicking the link allows the user to set a new password. The Admin can activate or deactivate any user account from the user management panel - deactivated accounts cannot log in but their data and order history are preserved. When a user logs out the token is cleared from the client.

Why it matters: Every feature in the system depends on knowing exactly who is logged in and what role they have. Without this, a student could access the admin panel, anyone could place orders on behalf of someone else, and the priority queue would have no basis to operate. Role identification at login is what makes the entire system possible.

2. Three-Tier Role System and Priority Queue

What it actually does: There are three operational roles - User (Student or Teacher/Faculty), Counter Staff, and Admin. Users who place orders are assigned a priority level based on their role at order submission time. Teaching Faculty receive Priority 1 and Students receive Priority 2. Counter Staff and Admin do not place orders.

Every confirmed paid order automatically enters the processing queue. The queue is sorted at all times by priority level ascending, then by submission timestamp ascending within the same priority level. If a Faculty member submits an order after three Students, the Faculty order moves ahead of all three automatically without any manual action.

The queue uses a composite scoring model rather than a binary priority number. The score for each order is calculated from three factors: role weight (Faculty = 1, Student = 2), a complexity weight based on page count and service type, and an aging score that increases the longer an order has been waiting. This means the queue balances role-based fairness with time-based fairness - a student order that has waited a long time will not be indefinitely blocked by a constant stream of faculty orders.

Starvation Prevention An aging mechanism runs every 10 minutes. Any order that has been waiting beyond a configurable threshold receives a temporary score boost that elevates it in the queue. This ensures no order waits indefinitely regardless of incoming higher-priority orders.

Per-Service Sub-Queues The queue is maintained as separate sub-queues per service type - Printing, Scanning, Photocopying, and Binding. Different equipment handles different job types simultaneously, so orders for different services can be processed in parallel. Each sub-queue maintains its own priority-sorted order.

Estimated Wait Time Formula The estimated wait time shown to users is calculated using exact per-order workload summation - not a rolling average. At the moment an order is created and confirmed, the system computes and stores an estimatedDuration value directly on the order document using the order's own parameters and the current admin-configured time values read live from the

database. The total wait time for a queued order is the sum of the estimatedDuration of every order ahead of it in the same service sub-queue. No averaging is involved at any stage. The time values used in the formulas are set by the admin from the system settings panel and can be updated at any time - changes apply to new orders only and never affect the stored estimatedDuration of orders already in the queue.

Service	Base Time Unit	Formula Applied
Printing	Admin-configured time per page	pages x copies x time per page (admin-configured)
Photocopying	Admin-configured time per page	pages x copies x time per page (admin-configured)
Scanning	Admin-configured time per page	pages x time per page (admin-configured)
Binding	Admin-configured fixed duration per binding type	sets x fixed duration per binding type (admin-configured per type)

This figure updates in real time via Socket.io whenever an order ahead is completed, cancelled, or a new higher-priority order is inserted ahead. Because each order carries its own stored `estimatedDuration`, the recalculation is a simple summation of fixed values - no estimation or averaging occurs at any point.

Token Number Generation Every order that enters the queue is assigned a formatted token number - PR1001 for printing, SC1001 for scanning, PH1001 for photocopying, BD1001 for binding - generated by counting today's orders for that service type and incrementing. The token number is always visible on the order tracking page, in notifications, and on the printed receipt.

Admin and Staff Queue Controls: The Admin can manually reposition any order in the queue by dragging and dropping. This action is logged in the admin activity log with a mandatory reason. Socket.io broadcasts all queue changes - new orders arriving, orders being processed, manual overrides - to all connected staff and admin dashboards in real time.

Why it matters: A first-come-first-served queue ignores the real-world dynamics of a campus where faculty time is more constrained than student time. The composite priority system handles this automatically. The token number and live wait time mean users can return to their work and come back only when their order is nearly ready.

3. Document Upload System with AI Analysis

What it actually does: Users upload their document by clicking the upload area or dragging and dropping. The system accepts PDF, DOC, DOCX, JPG, and PNG files up to 25MB. The backend receives the file through Multer middleware and validates the actual MIME type - not just the file extension - to confirm the file is genuinely the format it claims to be. A malicious file renamed with a PDF extension is caught and rejected at this stage.

Once validated, the file is sent to Cloudinary cloud storage with a system-generated unique filename so two files with the same original name never collide. Files are stored with access control using Cloudinary signed URLs - only the uploading user, counter staff, and admin can access a file URL. Unsigned direct access attempts are rejected. Previously uploaded files are saved in a personal document library so users can reuse them for future orders without uploading again.

AI Document Analysis (Runs Immediately After Every Upload) The backend performs three automated analyses on every uploaded file before the user proceeds to configure their order:

- Auto Page Count Detection - pdf-lib reads the uploaded PDF and returns the exact page count. The page count field in the order form is auto-filled instantly. The user does not need to count or enter page numbers manually. For multi-file uploads each file is counted separately and the totals are summed.
- Blank Page Detection - Each page is rendered as an image and analysed for pixel brightness mean and standard deviation using sharp and pdfjs-dist. A page with mean brightness above 240 and standard deviation below 10 is classified as blank. The user is shown exactly which pages appear blank and is given the choice to remove them from the order or include them anyway. This is a warning, not a block.
- Document Quality Check - Pages are analysed for overall brightness and print resolution. Pages with mean brightness below 50 are flagged as too dark. Pages with mean brightness above 240 but not classified as blank are flagged as too light or faded. Pages with dimensions below 150 pixels in either direction are flagged as low resolution. A summary panel shows each issue with the specific page number and a description. The user can still proceed - warnings are informational only.

The analysis runs once per upload and typically completes in 3 to 5 seconds for a standard 34- page document. A progress indicator is shown. Once analysis is complete, the user sees a results summary before proceeding to configure their order.

File Cleanup Policy Files uploaded through the Document Tools free utility (resize, compress, convert) are not stored permanently. They are processed server-side, returned to the user for download, and all temporary files are deleted immediately after. Files attached to orders are retained in Cloudinary for the lifetime of the order record. Cancelled order files are flagged for periodic cleanup.

Why it matters: Physical document handling is the biggest source of errors in a manual reprography centre - files get misplaced, USB drives get corrupted, and originals get damaged. Digital upload with cloud storage eliminates all of this. The AI analysis catches problems before printing - a blank page or low-resolution document is identified before the user pays, not after.

4. Document Processing Services

What it actually does: RAS supports five service types. Four - Printing, Photocopying, Scanning, and Binding - require payment and enter the processing queue. The fifth - Document Conversion - is completely free and does not enter the queue. All four paid services go through a 3-step order wizard. Document conversion is a standalone instant utility.

Service Key Parameters

Queue Payment Printing Copies, Color/B&W, Paper size (A4/A3/Legal), Page range, Single/Double- sided

Photocopying Copies, Color/B&W, Paper size, Reduction or Enlargement p Scanning Resolution (DPI), Output format (PDF/JPG/PNG), Color/B&W Yes Yes

Binding Type (Spiral/Comb/Perfect/Hardcover),

3-Step Order Wizard (All Paid Services)

Step 1 - Upload Document: User uploads file. AI analysis runs automatically - page count fills in, blank and quality issues are flagged.

- Step 2 - Configure Service: User selects all parameters for the chosen service. For Printing and Photocopying, a Print Instructions box appears below the standard settings. The user can type any page-specific or job-specific instructions in plain text - for example: Page 1 colour, pages 2 to 10 black and white. Page 5 double sided. Do not staple. Handle cover page carefully. This field is optional; if left blank, the global settings selected above apply to all pages. The instructions are saved directly against the order and cannot exceed 500 characters. The AI cost optimisation suggestion also appears here if applicable.
- Step 3 - Review and Confirm: Full itemised cost breakdown is displayed. Any print instructions entered in Step 2 are shown as a read-only summary so the user can verify them before confirming. User confirms before any payment is requested.

Document Conversion bypasses all three steps. The user uploads a file, the server converts it instantly using LibreOffice or mammoth, and the output file is immediately available for download. No payment, no token, no queue entry, no notification.

Why it matters: Covering all four common reprography services under one system means students and staff have a single place for every document need on campus. The free conversion utility is a genuine campus-wide benefit - students constantly need to convert files for submission and currently rely on random online tools with no privacy guarantees.

5. Document Tools - Free Utility Suite

What it actually does: A dedicated Document Tools page provides four free utility functions available to all registered users at any time. These process files instantly without payment, queue entry, or notification. They are separate from the paid order workflow.

Tool What It Does Accepted Formats Library PDF to Word / Word to PDF Converts between document formats instantly PDF, DOC, DOCX LibreOffice / mammoth Image Resize Change image width and height to specific dimensions JPG, PNG, WEBP sharp Image Compress Reduce file size by setting an output quality percentage JPG, PNG, WEBP sharp PDF Page Resize Scale all PDF pages to A4, A3, Legal, or custom dimensions PDF pdf-lib

Files processed through Document Tools are not stored permanently. The server processes the file, returns the output for immediate download, and deletes all temporary files. Maximum file size is 25MB, consistent with the main upload limit.

Why it matters: Students constantly need to compress large images before submission portals, resize scanned documents, or convert file formats. Having all of these available inside the same platform they use for printing makes RAS the complete campus document solution - not just a print ordering system.

6. Multi-Document Orders

What it actually does: Users can attach multiple files to a single order. This is most relevant for binding jobs - a bound document often consists of several separately prepared files - and for batch printing where different documents need to be printed under one transaction.

Each file in a multi-document order is uploaded and analysed individually by the full AI analysis pipeline. Blank and quality warnings are raised per file. Page counts from all files are automatically summed to produce the total page count used for cost calculation. The order review screen (Step 3) shows a per-file breakdown of pages and any flagged issues. All files are stored on Cloudinary linked under the same order reference ID.

Why it matters: A single-file-per-order constraint does not reflect how real binding or bulk print jobs work. Multi-document support removes a significant friction point for the most common complex order types.

7. Smart Cost Estimation with AI Optimisation

What it actually does: When the user reaches Step 3 of the order wizard the system fetches the current service pricing from the database and calculates the total cost. Pricing changes made by the admin take effect immediately for new orders. Confirmed orders always retain the price at which they were confirmed - price changes never affect existing orders.

Cost Formula Base cost equals pages multiplied by copies multiplied by the price per page for that service type. If colour output is selected a colour surcharge per page is added on top. For binding the cost is per set. If a coupon code has been applied the discount is subtracted last. The breakdown shown to the user itemises every component - base amount, colour surcharge if applicable, coupon discount if applied, and final total - before the user confirms anything.

AI Cost Optimisation Suggestion After the document analysis in Step 1, if the system detects that some pages have heavy colour content and others are predominantly text, it calculates the cost of a split printing strategy - colour pages printed in colour, text-heavy pages in black and white - and shows the potential saving alongside a one-click Apply button. For example: printing pages 1 to 8 in colour and pages 9 to 33 in black and white saves Rs.52. The user can apply the suggestion or ignore it and proceed with their original selection.

Why it matters: Showing the full cost breakdown before payment - not after - is the baseline expectation for any honest e-commerce system. The AI optimisation suggestion adds genuine monetary value for users printing large mixed-content documents.

8. Payment System

What it actually does: After reviewing the cost at Step 3 the user chooses between two payment methods. Document Conversion orders skip this step entirely. The payment amount is always fetched from the server at the time of checkout - the frontend cannot manipulate the price.

Online Payment via Razorpay A Razorpay popup appears where the user can pay using UPI, debit or credit card, net banking, or a mobile wallet. After the user completes payment, Razorpay sends the payment ID, order ID, and signature to the frontend. The frontend passes these three values to the backend. The backend independently recreates the expected signature using the HMAC-SHA256 algorithm with the Razorpay secret key and compares it to the received signature. If they match the payment is confirmed genuine and the order enters the queue. If they do not match the payment is marked as failed and the order does not proceed. A faked payment response cannot pass this verification.

Pay at Counter The user confirms the order and a record is created with status cash-pending. The order enters the queue and counter staff can see it is flagged as awaiting cash collection at pickup. When the user arrives and pays, the staff member marks the cash as collected and the payment status updates.

Payment Retry and Failure Handling If an online payment fails midway or is abandoned, the order is held in a payment-pending state for 15 minutes. The user sees a Retry Payment button and can attempt payment again without re-entering their order configuration. After 15 minutes the held order is automatically cancelled and the user is notified. A 2-minute warning notification fires before the auto-cancel deadline. Duplicate payment prevention is enforced through Razorpay's order ID system - the same Razorpay order ID cannot be successfully charged more than once.

Refunds The Admin can initiate refunds for cancelled paid-online orders directly through the Razorpay refund API from the admin panel. Cash payment orders are flagged for manual refund handling. All payment records are stored permanently with transaction ID, method, amount, timestamp, and status - including failed and refunded transactions.

Digital Receipt After a successful payment a digital receipt is immediately available for download. For completed orders a full receipt is also accessible from the order history page. Receipts are generated as formatted PDFs using pdfkit and contain the order ID, token number, user details, service specifications, full cost breakdown, payment method, transaction ID, and timestamp.

Why it matters: Cash-only payments at a reprography counter cause queues, change disputes, and untracked revenue. Online payment removes all three problems. The Pay at Counter option ensures users who prefer cash are not excluded. HMAC verification means no one can fake a payment confirmation and get their order processed for free.

9. Order Editing and Cancellation

What it actually does: Users can edit or cancel an order at any point before counter staff move it to Processing status. Once a staff member clicks Start Processing the order is locked - no further changes are permitted by the user. This ensures the document being processed is exactly what was configured and paid for.

When a user cancels a confirmed paid-online order the Razorpay refund API is triggered automatically. The user receives a notification confirming the cancellation and the initiation of the refund. Refund processing time follows Razorpay's standard timelines and is displayed in the notification.

Why it matters: Real users make mistakes - wrong page range, wrong paper size, wrong number of copies. Allowing edits and cancellations up to the point of physical processing is the correct operational design and is a commonly tested question in project evaluations.

10. Order Tracking

What it actually does: After placing an order the user can view its status at any time from their dashboard or the My Orders page. Every order passes through a defined sequence of statuses. Each status change is recorded in a status history array inside the order document in MongoDB, with a timestamp and the ID of the person who made the change - whether that is the system, a staff member, or the admin.

Status	Meaning	Who Sets It
Pending	Order placed, payment not yet confirmed	System (automatic)
In Queue	Payment confirmed, waiting to be processed	System (automatic)
Processing	Counter staff has started working on the order	Counter staff
Ready for Pickup	Physical output is ready at the counter	Counter Staff
Completed	User has collected the output	Counter Staff
Cancelled	Order cancelled by user, staff, or admin	User / Staff / Admin

On the tracking page the user sees a visual timeline showing all status transitions in chronological order with full dates, times, and the actor who made each change. The current queue position and estimated wait time are displayed while the order is In Queue status. Both figures update in real time via Socket.io - the page does not need to be refreshed. The token number is always visible on the tracking page and in all related notifications.

Why it matters: The single most frustrating experience at a manual reprography centre is not knowing if an order is ready yet. Real-time status tracking eliminates this entirely - users know exactly what is happening with their order at every moment without visiting the counter.

11. Partial Order Handling

What it actually does: In the event of equipment failure mid-job - for example a paper jam at page 30 of a 50-page print job - counter staff can mark the order as Partial Completion rather than fully cancelled or fully completed. The staff member enters the number of pages successfully completed. The system recalculates the charge for the completed portion only and updates the payment record accordingly. The remaining pages are automatically created as a follow-up order that re-enters the queue at the original user's priority level with a new token number. The user receives a notification describing the partial completion, the adjusted charge, and the new follow-up token number.

Why it matters: Equipment failures at print shops are a real operational occurrence. Without partial order handling the only options are to charge the user for work not done or to restart the entire job. Partial completion handles this correctly and fairly.

12. Slot Booking and Preferred Pickup Time

What it actually does: When placing an order, users can optionally select a preferred pickup time window. Available windows are Morning (9 AM to 12 PM), Afternoon (12 PM to 3 PM), and Evening (3 PM to 5 PM). This is a preference indication - the priority queue still governs the actual processing order. The slot selection does not guarantee a specific time; it is used as scheduling guidance.

If a selected time window is already heavily booked relative to expected throughput, the system suggests an alternative window and shows the expected congestion level. Users who do not select a slot proceed through the standard queue without any disadvantage. The preferred pickup window is shown as an informational tag on each order card in the counter staff queue panel, helping staff anticipate when users are likely to arrive. A pickup distribution summary on the staff dashboard shows expected counter traffic across the day.

Why it matters: Slot booking reduces counter crowding during peak hours - a persistent problem at campus print centres around exam periods - without disrupting the core priority queue logic.

13. Counter Staff Dashboard

What it actually does: Counter staff log in with their staff credentials and land on a focused operational dashboard. This interface is completely separate from the admin dashboard and shows only what is needed to process orders efficiently. The counter staff role cannot access analytics, pricing, user management, financial records, or any administrative function.

Dashboard Components

Live Queue Panel - all waiting orders sorted by priority then time, showing token number, user name and role, service type, page count, submission time, and preferred pickup slot if

selected. If the user entered print instructions, a highlighted note icon appears on the order card. New orders appear in real time via Socket.io. - Print Instructions Display - when a user has entered print instructions for an order, those instructions are shown as a prominently highlighted panel on the order detail view - visually distinct from the standard specifications so staff cannot miss them before starting the job.

Example: Page 1 colour, pages 2 to 10 black and white. Do not staple. The full instruction text is also printed on the internal job sheet if the admin has enabled job sheet printing. - Current Order View - clicking any order opens the full details including the Cloudinary document link, all service specifications, and payment status. - Order Processing Controls - Start Processing, Ready for Pickup, Complete, and Partial Completion buttons with confirmation prompts. Each action updates the order status and fires the corresponding notification to the user. - Cash Collection - Pay at Counter orders are flagged with a cash-pending indicator. Staff clicks Mark Cash Collected when the user pays at the

counter. - Inventory Quick View - current stock levels for paper types and toner with colour-coded low-stock indicators. - Complaint Messages - active complaint threads appear in a side panel. Staff can read and reply without leaving the dashboard. - Completed Today - running count of orders processed in the current session.

Why it matters: Counter staff are the operational core of the reprography centre. A clean, focused dashboard with only the controls they need - and none of the administrative complexity - lets them process orders quickly without errors or distractions.

14. Admin Dashboard

What it actually does: The admin logs in and lands on a comprehensive management dashboard. The admin interface is entirely separate from the staff dashboard and is inaccessible to staff accounts. Both frontend routing and backend middleware enforce this separation independently.

Summary Cards Total orders today, orders in queue, orders currently processing, orders ready for pickup, today's revenue collected, number of open complaints, and a count of inventory items currently below their minimum threshold.

User Management A searchable and filterable table of all registered users. The admin can view any user's full profile, complete order history, and total spend. Counter Staff accounts are created from this

section. Any account can be activated or deactivated instantly - deactivated accounts cannot log in but all their data is retained.

Pricing Management All service types are listed with their current price per page and colour surcharge. The admin edits any figure and saves - the change takes effect immediately for all new orders. Existing confirmed orders are never affected by price changes after confirmation.

Transaction History Complete record of every payment in the system - online and cash - filterable by date range, payment method, status, and user. Refund initiation for eligible cancelled orders is available directly from this section through the Razorpay refund API.

Staff Performance Table showing each staff member's daily order processing count, average processing time per order type, and shift activity timeline. This data is derived from the timestamps on status change records and is linked to each staff account ID.

Queue Override The admin can manually reposition any order in any sub-queue by dragging and dropping. This action requires a reason to be entered and is immediately logged in the activity log with the reason, the admin's ID, the affected order, and the timestamp.

Why it matters: Without a proper management dashboard the admin is operating blind - processing orders by feel, keeping prices in memory, and having no record of daily output. The admin dashboard transforms the reprography centre from a manual guessing game into a structured, data-driven operation.

15. Admin Activity Logs

What it actually does: Every sensitive administrative or staff action is recorded in an append-only activity log collection in MongoDB. Log entries cannot be modified or deleted. Each entry captures the action type, the account that performed it, a description, the affected record ID, and a timestamp.

Example Log Entry Pricing Change Admin updated Colour Printing price from Rs.3.00 to Rs.3.50 per page Inventory Update Staff updated A4 Paper stock from 50 to 2000 sheets User Deactivation Admin deactivated account for student@college.edu

Example Log Entry Queue Override Admin repositioned PR1008 from position 4 to position 1.
Reason: Faculty emergency request Complaint Resolution Staff marked complaint CMP1003 as Resolved Refund Initiated Admin triggered refund of Rs.214 for order PR1001 via Razorpay Staff Account Created Admin created Counter Staff account for staff2@college.edu

They serve as the complete audit trail for the system - for accountability, dispute resolution, and financial reporting.

Why it matters: In any system where multiple people can change pricing, inventory levels, and order statuses, an immutable audit trail is not optional - it is the foundation of accountability. Activity logs answer the question of who changed what, when, and why.

16. Inventory Tracker with Demand Forecasting

What it actually does: Counter staff and admin manage physical resource levels through the Inventory page. Stock is updated manually when supplies are replenished. Every update is logged in the activity log with the previous value, new value, timestamp, and the staff member who made the change.

Resource Category Items Tracked Unit Paper A4, A3, Legal Sheets Toner/Ink Black, Cyan, Magenta, Yellow Percentage remaining Binding Materials Spiral rolls, Comb strips, Covers Units / Sets

Each resource has a configurable minimum threshold. When any item falls below its threshold a red warning banner appears on both the staff and admin dashboards and an in-app notification is sent to the admin. Orders in the queue that require a critically low resource are flagged with a warning icon on their order card in the staff queue panel.

Demand Forecasting Based on historical order data the system calculates a rolling daily usage rate for each consumable resource and projects how many days of stock remain at the current consumption pace. For example: Black toner is projected to run out in 2 days at the current usage rate. Exam

period patterns from previous months are used to flag weeks where unusually high consumption is expected. These forecasts are shown on the inventory page and as a summary card on the admin dashboard.

Why it matters: Running out of A4 paper or toner mid-day at a campus print centre is a significant operational failure. Demand forecasting gives the admin time to restock before a shortage occurs rather than discovering it when a job fails.

17. Reports and Analytics

What it actually does: The Reports section is accessible only to the admin. It runs MongoDB aggregation queries against the orders and payments collections to produce summarised analytics. The admin selects a date range and the report type.

Report What It Shows Format Daily Orders Orders per day broken down by service type Bar chart + data table Revenue Summary Total revenue per day and running monthly total Line chart + data table Service Breakdown Percentage of orders by service type Pie chart + data table User Activity Users ranked by order count and total spend Sortable table Department Analytics Orders and revenue aggregated by department Bar chart + table Peak Hours Analysis Order volume by hour of day across the selected range Heat map + table Staff Performance Orders processed and avg processing time per staff member Table Payment Breakdown Split between UPI, card, cash, wallet; failed rate Pie chart + table Predictive Load Expected order volume based on historical day patterns Forecast chart

All reports can be exported as CSV for spreadsheet analysis or as a formatted PDF report using pdfkit - a professionally laid-out document suitable for monthly management filing. Charts are always accompanied by plain data tables for users who prefer numbers over visuals.

Why it matters:

An admin who knows 70 percent of orders are printing and peak demand is between 10 AM and 12 PM can make data-driven decisions about equipment maintenance, staffing schedules, and pricing. Without analytics the centre is operated on instinct alone.

18. Notifications with Persistent History

What it actually does: A Notifications collection in MongoDB stores records with fields for user ID, title, message, type, read status, and creation timestamp. Whenever a significant event occurs the backend calls a createNotification helper that inserts a record into this collection. The bell icon in the navbar shows a live unread count badge that refreshes automatically. Clicking the bell opens a dropdown with the five most recent notifications. A full Notifications History page shows every notification ever received with options to mark individual items as read, mark all as read, or delete individual entries.

Trigger Event Who Is Notified Order confirmed and entered queue User - includes token number and current queue position Counter staff starts processing order User - In Progress notification with token Order marked Ready for Pickup User - Ready notification with token number Order cancelled User - includes reason if one was provided Higher-priority order pushes user back User - optional queue position update Payment retry window expiring (2 min warning) User - warns before auto-cancel fires Cash payment order placed User - reminder to bring payment to counter New order placed in system All Counter Staff and Admin New complaint raised All Counter Staff and Admin Complaint status changed User - includes new status Admin or staff replies in complaint chat User - new message notification Inventory item falls below threshold Admin - resource and current level named

Why it matters: Users should never have to manually check the system to find out what is happening with their order. Persistent notification history means every communication between the system and the user is permanently accessible - no important update is ever lost because the browser tab was closed.

19. Order History and Records

What it actually does: Every user has a My Orders page showing their complete order history. Each row shows the Order ID, token number, service type, date submitted, current status, and total amount paid. The table can be filtered by status and service type and sorted by date ascending or descending.

Clicking any order opens a detailed view showing the full status timeline with timestamps and actor names, all service specifications and parameters, print instructions entered by the user (if any), payment method and transaction ID, a direct link to the original uploaded document on Clouidinary, and for multi-document orders a list of all attached files with individual links.

Completed paid orders have a Download Receipt button that generates a formatted PDF using pdfkit. The receipt contains the order ID, token number, user name and college ID, service type and full specifications, itemised cost breakdown, payment method, Razorpay transaction ID (for online payments), and timestamp of payment confirmation.

The Admin's Order Management page shows the complete system-wide order history - every order ever placed by every user - filterable by date range, service type, department, user, payment method, and status. The admin can click any order to view full details and add internal notes for operational reference. All order records are permanent and cannot be deleted by anyone.

Why it matters: Students need proof of payment for reimbursement claims. Staff need to track department document processing. Admins need a complete audit trail for financial reporting. A permanent, searchable, filterable order history with downloadable receipts satisfies all three needs simultaneously.

20. Complaint and Messaging System

What it actually does: Users can raise a complaint from the Order Details page of any completed, processing, or ready-for-pickup order, or from a dedicated My Complaints page. Each complaint generates a unique token - formatted as CMP1001, CMP1002, and so on - using the same token generation pattern as orders.

The complaint lifecycle uses the same status history pattern as the order system, making it immediately familiar to anyone who has used the order tracking feature.

Raising a Complaint The user selects the order the complaint relates to, chooses a category from Print Quality, Binding Quality, Wrong Output, Delay, Payment Issue, or Other, writes a description of the problem, and optionally attaches a photo or file as evidence. All attachments are stored on Cloudinary under the complaint reference. On submission the complaint token is generated, the complaint is set to Open status, and all counter staff and the admin are notified.

Complaint Status	Lifecycle Status	Meaning	Who Sets It
Open	Complaint submitted, awaiting staff or admin response	System	In Progress
Staff / Admin	Resolved	Issue has been addressed - user can confirm or reopen within 48 hrs	Staff / Admin
Closed	Fully closed, no further action	System (after confirmation) / Admin	

Real-Time Chat Thread Every complaint has a persistent chat thread. Both the user and the assigned staff member or admin can send messages in that thread at any time. Messages support text and file or image attachments stored on Cloudinary. The chat is powered by Socket.io - messages appear on both sides instantly without any page refresh. The full message history is permanently stored and accessible from both the user's My Complaints page and the staff and admin complaint panel.

Staff and Admin Complaint Panel A dedicated Complaints section in both the staff dashboard and admin dashboard shows all complaints in a filterable table - by status, category, and date range. Summary cards show total Open, In Progress, and Resolved Today counts. Clicking any complaint opens the full chat thread alongside the complaint details and a status update dropdown.

Every status change is timestamped and logged. The admin can escalate, reassign, or close any complaint regardless of who raised or responded to it.

Why it matters: Print quality issues, wrong outputs, and binding defects are a regular occurrence in any physical reprography centre. Without a structured complaint system users have no way to formally document a problem and no visibility into whether it is being addressed. The complaint token system creates accountability - every issue is tracked from raising to resolution.

21. AI Chatbot Assistant

What it actually does: A floating chat widget is present on every page of the application. Users can type questions about services, pricing, their order status, queue length, or how to use any feature. The chatbot is powered by the Gemini API (Google Generative AI).

At the time of each user query the backend injects live contextual data from the MongoDB database into the prompt - current pricing for all service types, current queue length per sub-queue, accepted file formats, and available services. This means the chatbot answers with accurate, real-time information rather than static hardcoded responses. Questions about current prices or queue status are always correct because they are answered with live database values.

Example User Query How the Bot Answers What is the price for colour A3 printing? Reads live pricing from database, gives exact current figure How many orders are ahead of me? Reads live queue count, gives current position context What file formats can I upload? Returns configured accepted format list My payment failed - what should I do? Explains the retry flow and 15-minute window How do I raise a complaint? Walks through the complaint submission steps What is the difference between spiral and comb binding? Explains service options from knowledge base

The chatbot is scoped to RAS-related queries only. Off-topic questions are politely declined. No conversation history is stored between sessions - each query is stateless.

Why it matters: A chatbot with live database injection is more genuinely useful than a static FAQ. Every answer about pricing or queue status is accurate at the moment it is given. This also reduces the volume of routine questions reaching counter staff.

22. Search and Filtering

What it actually does: Every data table in the system supports search and filtering. This applies on both the user-facing side and the staff and admin panels. No table in RAS displays raw unfiltered data without a way to search or narrow results.

Table Searchable Fields Filter Options My Orders (User) Order ID, Token Number Status, Service Type, Date Range

Table Searchable Fields : Filter Options Order Management (Admin) Order ID, Token, User Name, College ID, Dept. Status, Service, Dept., Date Range, Payment Method User Management (Admin) Name, Email, College ID Role, Department, Account Status Transaction History (Admin) Transaction ID, Order ID, User Name Payment Method, Status, Date Range Complaints (Staff + Admin) Complaint Token, User Name, Order ID Status, Category, Date Range Activity Logs (Admin) Action type keyword, Staff/Admin name Action Type, Date Range Inventory Log (Admin) Item name Date Range, Updated By

Why it matters: A system with hundreds of orders, users, and transactions becomes unusable without search and filtering. These are not enhancement features - they are baseline usability requirements for any system that accumulates persistent data.

23. User Feedback and Rating System

What it actually does: After an order is marked as Completed, the user receives an in-app notification prompting them to rate their experience. The rating screen shows a 1-to-5 star selector and an optional free-text comment field. Ratings are stored against the order record in the database. Users who do not wish to rate can dismiss the prompt - rating is never mandatory.

The Admin can view aggregated ratings in the analytics section - average rating per service type, average rating per staff member, rating trend over time, and a filterable table of all individual reviews. Low ratings below a configurable threshold trigger an internal flag that appears in the admin dashboard for follow-up.

Why it matters: A reprography centre that has no feedback mechanism has no reliable way to identify quality problems or staff performance issues. The rating system creates a direct data link between completed orders and service quality, feeding into the analytics section and staff performance reports.

24. Backup and Recovery

What it actually does: RAS uses MongoDB Atlas as the database host, which provides automatic daily snapshots with a configurable retention period. Point-in-time recovery is available for the most recent period. In the event of data loss or corruption the database can be restored to any snapshot within the retention window without manual intervention.

Cloudinary handles file storage redundancy independently - uploaded documents are stored with geographic redundancy by default and are not dependent on the application server's availability. This means a server failure does not result in loss of uploaded documents.

For recovery scenarios the system follows a defined order: restore the MongoDB snapshot to the most recent clean point, verify Cloudinary file references against the restored database to identify any orphaned or missing file records, and redeploy the application from the version-controlled codebase. Routine order operations are stateless from the server's perspective - the database and Cloudinary hold all persistent state, so server replacement does not require data migration.

Why it matters: A system handling real financial transactions and document submissions for an institution has a responsibility to protect that data. MongoDB Atlas automatic backups and Cloudinary's storage redundancy provide this without requiring custom backup infrastructure.

26. Operating Hours and Shop Status

What it actually does: The Admin configures the reprography centre's operating hours from the admin panel - for example Monday to Friday, 9 AM to 5 PM. These hours are stored in the database and checked by the system whenever a user attempts to confirm a new paid order. Outside configured hours the order form remains accessible - users can upload documents, configure settings, and even pay - but a clear banner is shown throughout: 'The reprography centre is currently closed. Your order will be queued and processed when we open tomorrow at 9 AM.' Orders placed outside hours sit at the front of the queue ready for staff when they log in next.

The Admin can also toggle a manual Open/Closed switch at any time - for unplanned equipment failure, staff absence, or public holidays. When closed, a system-wide announcement banner appears on every user's dashboard. All users with orders In Queue receive a notification informing them of the pause. When the Admin reopens, the queue resumes from where it stopped - no orders are lost or need resubmitting. The staff dashboard shows the current Open/Closed status prominently at the top. Staff cannot mark orders as Processing when the centre is Closed, preventing accidental processing outside hours.

Why it matters: Without operating hours logic, a user placing an order at midnight would see it sit in queue with no explanation of when it will be processed. A clear closed indicator with a reopen time manages user expectations correctly and reflects how every real service centre operates.

27. Reorder - Repeat a Previous Order

What it actually does: On the detail page of any completed order the user sees a Reorder button. Clicking it creates a new draft order pre-filled with all settings from the original - the same document from the user's document library, the same service type, page range, colour setting, paper size, number of copies, and any print instructions. The user is taken directly to Step 3 of the order wizard to review the cost and confirm. No re-uploading or re-configuring is needed.

The cost at Step 3 reflects current pricing at the time of reorder - not the original price. If the admin has changed pricing since, the new price applies. If the original document has been deleted from the document library due to the file expiry policy, the system notifies the user and asks them to upload a new copy. The reorder creates a completely independent new order record with a new Order ID and token number.

Why it matters: Students who print the same notes, assignments, or study materials repeatedly across a semester currently have to reconfigure the entire order from scratch each time. Reorder eliminates this entirely. It is one of the highest-frequency real-world use cases in a campus reprography centre.

28. Document Preview Before Payment

What it actually does: After a document is uploaded in Step 1 and AI analysis completes, a preview panel displays rendered thumbnails of the first five pages directly in the browser using pdfjs-dist. For image files the full image is shown. For Word documents the server converts to PDF before rendering the preview. The preview is rendered client-side from the already-uploaded Clouldinary file - nothing additional is stored.

The preview lets the user confirm they uploaded the correct file and the correct version before configuring and paying. If the wrong document is shown, the user clicks Replace File and re- uploads without losing their wizard progress.

Pages flagged by the AI analysis - blank, too dark, too light, or low resolution - are highlighted in the preview with a coloured border and warning icon so the user can see exactly which pages triggered each alert. Once the user confirms the preview, they click Proceed to Step 2.

Why it matters:Printing the wrong file or the wrong version is the single most common and most frustrating user error in any print system. A visible preview before payment gives the user one final chance to catch the mistake before money changes hands and the physical job begins.

29. Pickup Verification via OTP

What it actually does: When counter staff marks an order as Ready for Pickup, the system generates a 4-digit OTP and sends it to the user through two channels simultaneously - as an in-app notification and as an email to the user's registered email address. The OTP is stored against the order record and is valid for 24 hours. If the user does not collect within 24 hours the OTP expires and a fresh one can be requested from the order tracking page, which invalidates the previous one.

When the user arrives at the counter they present the OTP to the staff member, who enters it into the collection confirmation field and clicks Verify and Complete. The system validates the OTP against the stored value. If it matches the order status moves to Completed. If it does not match the attempt is logged and the staff member sees an error. Three consecutive failed attempts lock the collection and require admin intervention to unlock, preventing brute-force guessing. For Pay at Counter orders, cash collection and OTP verification happen in one combined step.

Why it matters: Without pickup verification, anyone who knows another person's token number could collect their printout. For a campus system handling exam papers, project reports, and personal documents this is a genuine security concern. OTP verification ensures the physical output reaches only the person who ordered it.

30. Maximum Order Size Limits

What it actually does: The Admin configures maximum allowed values per service type from the Pricing Management section. These limits are enforced at Step 2 of the order wizard - if the user's configuration exceeds any limit, the Proceed button is disabled and a clear message is shown explaining the maximum and suggesting the order be split or the admin be contacted for a bulk arrangement.

Service Configurable Limits Suggested Default Printing Max pages per order, max copies per order 500 pages, 10 copies Photocopying Max pages per order, max copies per order 500 pages, 10 copies Scanning Max pages per order 100 pages

Service Configurable Limits Suggested Default Binding Max sets per order, max pages per set 5 sets, 500 pages per set

The Admin can grant limit overrides for specific users or departments from the user management panel - for example allowing a department head a higher page limit for a specific period.

All overrides are logged in the activity log. Additionally the Admin can set a maximum number of simultaneous In Queue orders per user and a maximum number of orders per user per day. These prevent any single user from flooding the queue and ensure fair access for everyone on campus.

Why it matters: Without order size limits a single user placing a 5000-page print job could block the entire queue for hours. Configurable limits protect fair access and are a standard feature of any multi-user shared service system.

31. User Profile Page

What it actually does: Every registered user has a Profile page accessible from the navbar. It is divided into three sections - Personal Information, Account Security, and Usage Summary.

Personal Information Displays the user's name, email address, college ID (read-only), department, role, and an optional phone number. The user can edit their name, department, and phone number. Email changes require re-verification through a confirmation link sent to the new address. College ID and role cannot be self-edited - changes require admin action.

Account Security Contains a Change Password form requiring the current password, new password, and confirmation. Bcrypt hashes the new password before storage. A Logout from All Devices button invalidates all active JWT tokens for the account - useful if the user suspects a compromised session on another device. Active sessions showing device type and last active timestamp are listed so the user can see where their account is currently signed in.

Usage Summary A read-only panel showing total orders placed, total amount spent, most used service type, and account creation date - giving users a personal overview without needing to browse the full order history.

Why it matters: A user profile page is a baseline expectation in any registered web application. Its absence would be immediately noticed. The Logout from All Devices feature and active session list elevate it from a basic profile to a security-aware one.

Feature 32 - Email Verification at Registration

What it actually does: When a new user completes the registration form and submits, their account is created in MongoDB with a verified status of false. The system immediately generates a secure random verification token, stores a hash of it against the user record, and dispatches a verification email to the registered address via Nodemailer. The email contains a single-use verification link that embeds the token as a URL parameter. Until the user clicks the verification link, their account exists but is restricted - they can log in but cannot place orders, upload documents, or access any paid feature.

A persistent banner on their dashboard informs them that their email is unverified and shows a Resend Verification Email button. When the user clicks the link, the backend extracts the token from the URL, hashes it, and compares it against the stored hash on the user record. If it matches and has not expired, the verified field is set to true and the restriction is lifted immediately. The user is redirected to their dashboard with a confirmation message.

If the token has expired or is invalid, the user is shown an error and offered the option to request a new link. The verification link expires after 24 hours. Requesting a new link invalidates the previous one - only one valid verification token exists per account at any time. Once verified, the email verification flow never repeats unless the user changes their email address, in which case the new address must be verified before it replaces the old one. Counter Staff accounts created by the Admin are marked as verified automatically at creation - they do not go through the email verification flow since the Admin has already confirmed their identity.

Why it matters: Unverified email addresses break notification delivery silently. A user who mistyped their email would receive no pickup OTP, no order confirmation, and no status updates - with no indication of why. Verification at registration catches this immediately, before any order is placed, ensuring every notification channel works correctly from the first interaction.

33. Pay at Counter - Uncollected and Unpaid Order Handling

What it actually does: Pay at Counter orders introduce two real-world failure scenarios that the system handles with automated timeout flows and dedicated order statuses. These scenarios are completely separate from the standard cancellation flow.

Scenario A - User Never Pays (Unpaid Timeout) When a Pay at Counter order is marked Ready for Pickup and the payment status is still Cash Pending, a configurable countdown timer starts - defaulting to 48 hours. At 24 hours the user receives a reminder notification and email: 'Your order PR1001 is ready for pickup and awaiting payment. Please collect within 24 hours or the order will be cancelled.' At 48 hours, if the cash has still not been collected by staff, the order is automatically moved to Expired status. The admin receives a notification listing all newly expired orders.

Staff see expired orders in a dedicated Expired tab on their dashboard, separate from the active queue. The physical printout is held at the counter for the configured holding period and then disposed of. Expired orders are recorded permanently in the order history with the reason 'Unpaid - timeout reached.'

Scenario B - User Pays But Never Collects (Uncollected Timeout) When an order - whether paid online or cash collected - is marked Ready for Pickup and the user does not collect within a configurable period (default 72 hours), the system triggers an uncollected flow. At 24 hours a first reminder notification and email are sent. At 48 hours a second reminder fires. At 72 hours the order is automatically moved to Uncollected status. For online-paid orders the admin decides whether to hold further or initiate a refund - the admin dashboard shows all Uncollected orders with the amount paid and the number of hours since Ready for Pickup. For cash-paid orders the status is recorded as Collected Revenue, Output Disposed. All timeout durations are configurable by the admin from the system settings panel.

Scenario C - Payment Disputed at Counter If a user arrives at the counter and disputes the amount shown - refuses to pay the full amount or questions the charge - the staff member can flag the order as Payment Disputed from the order detail view. This immediately raises an internal alert to the admin and creates a linked complaint record automatically. The order is held in the Payment Disputed status - it is neither completed nor cancelled. The admin reviews the dispute, communicates with the user through the complaint thread if needed, and resolves by either adjusting the amount, waiving the charge, or cancelling the order. All dispute resolutions are logged in the activity log.

Complete Order Status Map Including New Statuses

Status	Meaning	Who Sets It
Pending Order	Order placed, payment not yet confirmed	System
In Queue	Payment confirmed, waiting to be processed	System
Processing	Counter staff has started the job	Counter Staff
Ready for Pickup	Physical output ready at counter	Counter Staff
Completed	User collected, OTP verified	Counter Staff
Cancelled	Cancelled by user, staff, or admin	User / Admin
Expired	Pay at Counter order - user never paid within timeout period	System (auto)
Uncollected	Paid order - user never collected within timeout period	System (auto)

Status Meaning Who Sets It

Status	Meaning	Who Sets It
Payment Disputed	User disputed amount at counter - under admin review	Counter Staff
Partial Job	Job partially completed due to equipment failure	Counter Staff

Why it matters: Without timeout handling, unpaid and uncollected orders accumulate indefinitely in the system with no resolution. In a real campus reprography centre this means physical printouts pile up at the counter, queue records never close, and revenue figures are inaccurate. The timeout flow closes every order cleanly regardless of the user's behaviour after placing it.

34. Staff Operational Reporting System

What it actually does: The Staff Operational Reporting System is a dedicated internal channel for counter staff to raise operational issues directly to the admin. It is completely separate from the user-facing complaint system. Users never see staff reports and staff reports are never mixed with user complaints.

What Staff Can Report Report Category Examples Low Stock A4 paper running low, spiral binding rolls nearly finished, cover sheets depleted Low Ink / Toner Black toner below safe level, cyan cartridge nearly empty Equipment Fault Printer paper jam recurring, scanner producing skewed output, binding machine not heating Maintenance Required Printer needs cleaning cycle, scanner glass dirty, equipment due for servicing Other Counter area issue, safety concern, any operational matter not covered above

Raising a Report From the staff dashboard a Staff Reports button is always visible in the sidebar. Staff click it, select a category, optionally select the specific item (e.g. A4 Paper, Black Toner, Printer 1), write a description of the issue, and optionally select an urgency level - Normal or Urgent. Submitting the report generates a unique report reference number (RPT1001, RPT1002 etc.) and immediately notifies the admin via the admin dashboard and an in-app notification. If urgency is set to Urgent the notification is flagged differently and appears at the top of the admin's report list.

Admin Actions on Reports The admin sees all staff reports in a dedicated Operations Reports section on the admin dashboard, sorted by urgency then submission time. Each report shows the category, item, description, urgency, staff member who raised it, and current status. The admin must acknowledge every report - clicking Acknowledge sends an automatic notification back to the staff member: 'Your report RPT1001 has been seen by the admin.' This acknowledgement is mandatory - if any report remains unacknowledged for more than 2 hours a reminder notification fires to the admin. After taking action the admin marks the report as Resolved and optionally adds a resolution note - for example: 'A4 paper restocked - 2000 sheets added' or 'Printer engineer booked for Tuesday.'

Staff Visibility of Report Status Staff can see the full history of all reports they have raised from the Staff Reports page - the report reference, category, description, current status (Pending, Acknowledged, Resolved), and the admin's resolution note if one was added. This closes the feedback loop - staff always know whether their report has been seen and what action was taken. They are never left wondering if their concern was noticed.

Integration with Inventory When a staff member raises a Low Stock report for a specific inventory item and the admin subsequently updates that item's stock level in the Inventory Tracker, the system links the two records. The resolution note on the report is auto-populated with the new stock quantity. This creates a clear audit trail from the moment a staff member notices low stock to the moment the admin restocks.

Report History and Analytics All operational reports are stored permanently. The admin can view report analytics - most frequently reported issues, average time from report to resolution, most common fault categories. This data helps identify recurring equipment problems and informs maintenance scheduling decisions.

Why it matters: Without a structured reporting channel, staff communicate operational problems informally - verbally, via WhatsApp, or by writing notes. These communications leave no audit trail, can be missed, and have no accountability mechanism. A formal reporting system with mandatory acknowledgement and visible resolution status ensures every operational issue is recorded, tracked, and closed - not forgotten.

35. Security - Comprehensive Layer-by-Layer Implementation

What it actually does: Security in RAS is not a single feature bolted on at the end. It is a set of independent, layered controls applied at every stage of the system - from registration through to payment, file storage, and deployment. Each control addresses a specific, real attack vector. The security

implementation follows industry-standard practices organised into six categories: Authentication and Access Control, Input and Data Validation, Error Handling and Information Exposure, Payment and Transaction Security, Infrastructure and Deployment Security, and Dependency and Supply Chain Security.

Authentication and Access Control Security Control Implementation Attack Vector Addressed Bcrypt password hashing Salt factor 12 applied before any storage. Plain text password never touches the database. Password theft from database breach Password strength enforcement Minimum 8 characters, at least one uppercase, one number, one special character. Enforced at registration and password change. Real-time strength indicator on frontend. Weak passwords that are trivially guessable JWT token verification Every protected route passes through auth middleware. Expired, missing, or tampered tokens return

401. Token is never stored in localStorage - stored in httpOnly cookie to prevent XSS access.

Unauthorized API access, token theft via XSS Role-based endpoint guards Second independent layer after JWT check.

36. Decentralised Smart Kiosk Mode

What it actually does: RAS supports a self-service guest mode for walk-in users who do not have a registered account and do not wish to register. A clearly visible User as Guest button is displayed on the homepage as a separate entry point, distinct from the standard login and register flow. This allows any walk-in user to access the system immediately without prior registration. Guest Authentication via Email OTP The guest clicks user as Guest and enters their email address. The system generates a 6-digit OTP and dispatches it to that address via Nodemailer. The guest enters the OTP on screen.

On successful verification, the backend creates a temporary guest session token, stores it in an httpOnly cookie using the same mechanism as registered user sessions, and grants the guest access to the order placement interface. No account is created, no password is set, and no profile is stored. The session is identified internally by the guest email address and a system-generated guest session ID. Guest Session Experience

Once authenticated, the guest accesses the full order placement interface identical to that of a registered user. The guest session interface is clearly labelled with a persistent Guest Session indicator visible throughout the session so the user is always aware of their session type. The guest can upload documents, go through the complete 3-step order wizard, receive AI document analysis including auto page count, blank page detection, and document quality check, view the document preview, and configure all service parameters exactly as a registered user would.

Payment Guest users pay exclusively at the counter. The online payment option via Razorpay is not available in guest sessions. When the guest confirms their order, it is created with payment status Cash Pending and enters the processing queue immediately. The counter staff sees the cash-pending indicator on the order card and collects payment when the guest arrives for pickup. Queue Priority Guest orders are always assigned Priority 2 - the same priority as registered students - regardless of who the guest is. Since guest users are unauthenticated and their identity cannot be verified, granting self-declared role-based priority would allow any guest to claim Faculty priority and jump ahead of registered students. Priority 2 is fair, safe, and requires no verification.

Registered faculty members who require Priority 1 must use their registered account. Guest Tag on Staff Queue Panel Guest orders are visually distinguished from registered user orders in the counter staff queue panel. Every order card originating from a guest session displays a clearly visible Guest tag alongside the token number and service details. This tag is rendered by the frontend by reading the isGuest flag stored on the Order document - no additional database query is required. The Guest tag serves three operational purposes: it tells staff that payment is always cash, that there is no registered account behind the order for dispute escalation, and that the interaction at the counter may require more assistance than a regular registered user order.

Session Termination and Data Retention : The guest session terminates automatically when the order reaches Completed status - that is, after the staff member verifies the OTP and marks the order as collected. The session also terminates after a configurable inactivity timeout if the guest does not complete an order within that period. The guest can also manually end their session at any time. On termination, the session token is cleared from the cookie and the guest is returned to the homepage. All guest order records, payment records, and uploaded document references are retained permanently in MongoDB after session termination, flagged with the isGuest marker.

These records are accessible only to admin and counter staff - for audit, dispute resolution, and financial reporting purposes. The guest cannot access these records after logout as they have no persistent account. Guest order history is never visible on any user-facing page.

What the Guest Cannot Do

Pay online via Razorpay - cash at counter only

Access order history from previous guest sessions

Use the document library - each guest session starts with a fresh upload

Raise complaints - guest orders can only be queried by admin or staff after session

Why it matters: A registration barrier at the point of first use loses walk-in users entirely. Kiosk Mode eliminates that barrier - any person who walks up to the reprography counter can place an order in under two minutes using only their email address. The full AI analysis pipeline runs for guest uploads just as it does for registered users, so the quality and accuracy of service is identical. The Guest tag on the staff queue panel ensures counter staff are always prepared for a cash transaction without needing to check the order details. This feature bridges the digital-physical gap and makes RAS accessible to the entire campus population, not just those who have registered in advance.

Kiosk Mode: Lifecycle Notifications for Guest Sessions Since guests have no persistent account and no in-app notification bell, all order lifecycle notifications are delivered exclusively via email to the address used during guest authentication. Socket.io in-app notifications are not sent for guest sessions - the guest has no persistent dashboard room to receive them.

The following events trigger email notifications to the guest's authentication email address:

Trigger Event Email Sent to Guest Order confirmed and entered queue Confirmation with token number, service summary, and estimated wait time Counter staff starts processing "Your order [token] is now being processed" Order marked Ready for Pickup "Your order [token] is ready - your pickup OTP is [OTP]" Order cancelled by staff or admin Cancellation notice with reason if provided Inactivity timeout approaching Warning that session will expire if no action is taken The pickup OTP (Feature 29) is sent to the same guest authentication email - this is the explicit delivery channel for guests in place of the dual in-app and email delivery used for registered users. If the guest keeps their browser tab open after placing the order, they can see live queue position and estimated wait time on the order confirmation screen, which updates via Socket.io for the duration of that browser session. If the tab is closed, the email notifications are the only communication channel.

Browser close and re-authentication: If a guest closes the browser after placing an order, they can re-authenticate by clicking Use as Guest again, entering the same email address, and completing a new OTP verification. The backend matches the incoming email to any active guest session associated with that address and restores the session token in a new httpOnly cookie. The guest is returned to their active order status screen. This re-authentication works only while the guest session is still active - it cannot be used to access completed or expired guest sessions. OTP rate limiting for guest authentication: The guest OTP request endpoint is rate limited at 5 requests per email address per 30-minute window and 10 requests per IP address per 15-minute window.

Both limits apply independently - hitting either returns a 429 response with a message indicating when the next request is permitted. This prevents the endpoint from being used to spam arbitrary email addresses.

37. PWA Architecture

RAS is not only a normal website. It is implemented as a Progressive Web App so that students can use it like a mobile application on Android or other modern browsers. The app includes a web manifest, installable icons, mobile meta tags, a service worker, offline fallback support, and browser push notification handling. This makes RAS more suitable for campus use because students often check order status from phones rather than desktop systems.

Installable App Shell The PWA install metadata is defined. During activate it deletes old cache versions and claims clients. During fetch it watches navigation requests and returns offline.html if the network request fails. **Service Worker Event RAS Behaviour** install Caches offline page, icons, and manifest, then calls skipWaiting. activate Deletes old caches and claims currently open RAS tabs. fetch If navigation fails because the user is offline, serves offline.html. push Receives backend push payloads and displays browser notifications.

Why This Feature Was Needed : A reprography system is most useful when students can quickly check it from their phones. PWA support means RAS can sit on the home screen like a regular app, show a branded offline page instead of a browser error, and receive push alerts when important events happen. This reduces repeated counter visits and makes the digital workflow feel closer to a native campus utility.

38. Friend Chat System

RAS includes a separate friend and chat system for students and faculty. This is different from the complaint chat described earlier in the original document. Complaint chat is a support channel between users and staff/admin. Friend chat is a collaboration channel between campus users. It supports friend search, friend requests, accepted friend lists, one-to-one messages, group chats, media messages, voice messages, read receipts, typing indicators, and real-time Socket.io updates. Friendship Layer The Friendship model stores requester, recipient, status, createdAt, and updatedAt. Its compound unique index on requester + recipient prevents duplicate friend request records between the same two users. The friendship routes allow Student and Faculty users to search people, send requests, respond to requests, fetch friends, fetch pending requests, and remove friends. Staff and Admin users are blocked from student/faculty friend search and request actions.

One-to-One and Group Messages The Message model is shared by one-to-one messages and group messages. For a direct message, recipientId is set and groupId is null. For a group message, groupId is set and recipientId is null. A pre-validate hook ensures that a message cannot exist without either a recipient or a group. Message records can contain text, image/pdf media, voice audio, read status, and optional splitCardData for split payment cards

Why This Feature Was Needed : Group projects are common in a college environment. Without a friend layer, students would need to manually select or identify other users for every collaboration. By adding friends and chat, RAS creates a trusted social layer inside the reprography workflow. The same layer later supports group orders and split payments because users can send payment requests to known friends instead of arbitrary strangers.

39. Split Requests and Live Split Cards

RAS supports group-order cost sharing through split requests and live split cards. This feature extends the normal order and wallet system. A creator can create a group order or custom split, assign amounts to friends, and send split requests. Each participant receives a notification and can accept or decline. If accepted, the participant's wallet is debited and the creator's wallet is credited in a MongoDB transaction.

Data Model - The GroupOrder model stores the orderId, createdBy, totalAmount, participants, description, and status. Each participant contains userId, amount, walletStatus, and paidAt. The SplitRequest model stores groupOrderId, from, to, amount, status, triggeredFrom, optional groupChatId, createdAt, and resolvedAt.

This separates the group-level payment plan from each individual participant's request. Model Purpose Important Fields Order The actual print/scan/binding order. isGroupOrder, groupOrderId, finalCost, tokenNumber, status. GroupOrder The shared payment plan linked to an order or custom split. orderId,

createdBy, totalAmount, participants, status. SplitRequest One request sent to one participant. from, to, amount, status, triggeredFrom, groupChatId. Wallet Current balance for participant and creator. userId, balance. WalletTransaction Append-only ledger for debit and credit entries and current user's participant entry. Participants see Pay or Decline actions only for their own pending split. When the backend emits splitCardUpdate, ChatWindow or GroupChatWindow refreshes the card without a page reload.